

Nama: Hans Malvin Djojo

<https://github.com/hansmalvin/LastMile>

Step-by-Step Instructions

Step 1: Define Mongoose Schemas and Models

Create the User.js model:

Inside the models directory, create a file named User.js and add the following code:

```
javascript
// models/User.js
const mongoose = require('mongoose');
const userSchema = new mongoose.Schema({
  username: { type: String, required: true, unique: true },
  email: { type: String, required: true, unique: true }
});

module.exports = mongoose.model('User', userSchema);
```

This defines a simple user schema with username and email fields, both required and unique.

Create the Post.js model:

Inside the models directory, create a file named Post.js and add the following code:

```
javascript
// models/Post.js
const mongoose = require('mongoose');
const postSchema = new mongoose.Schema({
  title: { type: String, required: true },
  content: { type: String, required: true },
  author: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
}, { timestamps: true });

module.exports = mongoose.model('Post', postSchema);
```

This defines a post schema with title, content, and author fields. The author field is an ObjectId that references the User model. This is the crucial part for population. The timestamps: true option automatically adds createdAt and updatedAt fields to the schema.

Step 2: Implement Logger Middleware

Create the logger.js middleware:

Inside the middleware directory, create a file named logger.js and add the following code:

```
javascript
// middleware/logger.js
const logger = (req, res, next) => {
  console.log([`${new Date().toISOString()}`] `${req.method} ${req.url}`);
  next(); // Call next to pass control to the next middleware or route handler
};
module.exports = logger;
```

This middleware logs the request method, URL, and timestamp to the console. The next() function is crucial; it passes control to the next middleware in the chain or the route handler.

Step 3: Create Post Routes

Create the posts.js route file:

Inside the routes directory, create a file named posts.js and add the following code:

```
javascript
// routes/posts.js
const express = require('express');
const router = express.Router();
const Post = require('../models/Post');
// GET all posts (with population)
router.get('/', async (req, res) => {
  try {
    const posts = await Post.find().populate('author', 'username email'); // Populate author field
    res.json(posts);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

module.exports = router;
```

This route retrieves all posts from the database and uses populate('author', 'username email') to replace the author ObjectId with the actual user document. The second argument to populate specifies which fields from the User document to include in the populated data (in this case, username and email).

Step 4: Configure the Express Application

Update app.js:

Replace the contents of app.js with the following code:

```
javascript
// app.js
const express = require('express');
const mongoose = require('mongoose');
const morgan = require('morgan');
const logger = require('./middleware/logger');
const postsRouter = require('./routes/posts');
const app = express();
const port = 3000;

// Connect to MongoDB
mongoose.connect('mongodb://127.0.0.1:27017/mern_middleware_population', {
  useNewUrlParser: true,
  useUnifiedTopology: true
})
.then(() => console.log('Connected to MongoDB'))
.catch(err => console.error('MongoDB connection error:', err));

// Middleware
app.use(express.json()); // Parse JSON request bodies
app.use(morgan('dev')); // HTTP request logging
app.use(logger); // Custom logger middleware

// Routes
app.use('/posts', postsRouter);

app.listen(port, () => {
  console.log(`Server is running on port ${port}`);
});
```

This code sets up the Express application, connects to MongoDB, registers middleware (including the custom logger and morgan for HTTP request logging), and mounts the posts route. `express.json()` is crucial to parse JSON data sent in request bodies (e.g., when creating new posts via POST requests).

Step 5: Create Seed Data (Optional, but recommended)

To effectively test the population, you'll need some initial data in your MongoDB database. You can create this data manually using MongoDB Compass or the MongoDB shell, or you can add code to your app.js to seed the database on application startup. Here's an example of how to seed the database programmatically:

Add seed data logic to app.js:

Modify your app.js to include the following code block after the MongoDB connection and before defining your routes:

```
javascript
// Seed data (optional)
const User = require('./models/User');
const Post = require('./models/Post');
const seedData = async () => {
  try {
    // Check if users already exist
    const existingUsers = await User.find();
    if (existingUsers.length === 0) {
      // Create users
      const user1 = new User({ username: 'john_doe', email: 'john.doe@example.com' });
      const user2 = new User({ username: 'jane_smith', email: 'jane.smith@example.com' });
      await user1.save();
      await user2.save();

      // Create posts
      const post1 = new Post({ title: 'First Post', content: 'This is the first post.', author: user1._id });
      const post2 = new Post({ title: 'Second Post', content: 'This is the second post.', author: user2._id });
      await post1.save();
      await post2.save();

      console.log('Seed data created successfully.');
```

```
    } else {
      console.log('Seed data already exists. Skipping seed operation.');
```

```
    }
  } catch (err) {
    console.error('Error seeding data:', err);
  }
};

seedData();
```

This code checks if any users already exist in the database. If not, it creates two users and two posts, linking the posts to their respective authors using the author field in the Post model.

Step 6: Run the Application

Start the server:

```
bash
npm start
```

This will start the Node.js server using nodemon. You should see the message "Server is running on port 3000" in your console, as well as "Connected to MongoDB" and potentially "Seed data created successfully." or "Seed data already exists. Skipping seed operation."

Step 7: Test the Endpoint

Access the posts endpoint:

Open your web browser or use a tool like curl or Postman to send a GET request to `http://localhost:3000/posts`.

- * Using curl:

```
bash
curl http://localhost:3000/posts
```

Validation & Expected Results

When you access the `/posts` endpoint, you should receive a JSON response containing an array of posts. Each post object should include the populated `author` field, containing the username and email of the corresponding user.

Example Output:

```
json
[
  {
    "_id": "64d15e1a2e9e8a3a8f7a4c31",
    "title": "First Post",
    "content": "This is the first post.",
    "author": {
      "_id": "64d15e1a2e9e8a3a8f7a4c2f",
      "username": "john_doe",
      "email": "john.doe@example.com"
    },
    "createdAt": "2023-08-07T14:35:30.000Z",
    "updatedAt": "2023-08-07T14:35:30.000Z",
    "__v": 0
  },
  {
    "_id": "64d15e1a2e9e8a3a8f7a4c32",
    "title": "Second Post",
    "content": "This is the second post.",
    "author": {
```

```
"_id": "64d15e1a2e9e8a3a8f7a4c30",
"username": "jane_smith",
"email": "jane.smith@example.com"
},
"createdAt": "2023-08-07T14:35:30.000Z",
"updatedAt": "2023-08-07T14:35:30.000Z",
"__v": 0
}
]
```

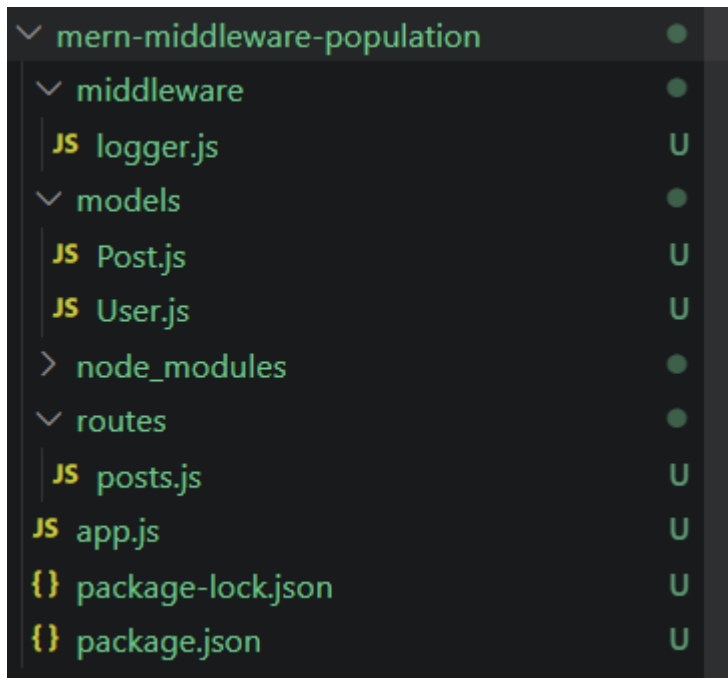
Environment Setup

```
mkdir mern-middlewre-population
cd mern-middlewre-population
npm init -y
npm install express mongoose morgan
npm install -D nodemon
```

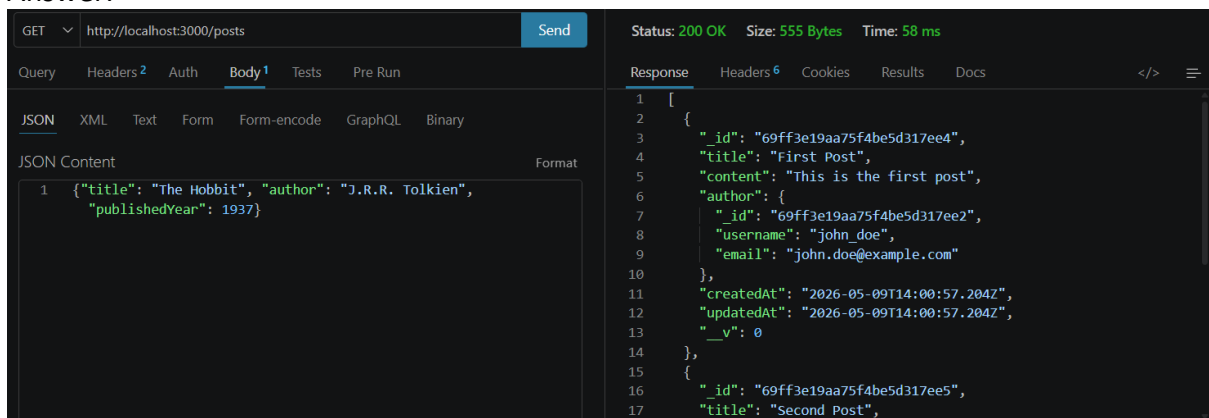
mern-middlewre-population/

```
├── models/
│   ├── User.js
│   └── Post.js
├── middleware/
│   └── logger.js
├── routes/
│   └── posts.js
├── app.js
└── package.json
```

```
"scripts": {
  "start": "nodemon app.js"
}
```



Answer:



The screenshot displays a web client interface (Postman) and a terminal window. The web client is configured with a GET request to `http://localhost:3000/posts`. The request body is a JSON object: `{ "title": "The Hobbit", "author": "J.R.R. Tolkien", "publishedYear": 1937 }`. The response status is 200 OK, with a size of 555 Bytes and a time of 58 ms. The response body is a JSON array containing two objects, each representing a post with an ID, title, content, author, createdAt, and updatedAt.

```
13 { "_v": 0
14 },
15 {
16   "id": "69ff3e19aa75f4be5d317ee5",
17   "title": "Second Post",
18   "content": "This is the second post",
19   "author": {
20     "id": "69ff3e19aa75f4be5d317ee3",
21     "username": "jane_smith",
22     "email": "jane.smith@example.com"
23   },
24   "createdAt": "2026-05-09T14:00:57.206Z",
25   "updatedAt": "2026-05-09T14:00:57.206Z",
26   "_v": 0
27 }
28 ]
```

The terminal window shows the command `npm start` being executed. The output indicates that the server is running on port 3000, connected to MongoDB, and seed data has been created. The terminal also shows the response of the GET /posts request, which is a 200 status with a response time of 32.692 ms and a size of 555 bytes.

Middleware acts as a strategic intermediary in the Express backend, intercepting incoming requests to perform essential tasks like authentication, logging, or data validation before they reach the final route handler. By functioning as a series of "gatekeepers," middleware ensures that only secure and properly formatted information is allowed to proceed, which centralizes logic and significantly streamlines error handling.

Population is a Mongoose feature designed to handle relationships between different data collections in a NoSQL environment like MongoDB. Instead of duplicating entire data sets across multiple documents, developers store simple references (IDs) to link collections together; population then automatically replaces these IDs with the actual linked data during a query, functioning similarly to a "JOIN" in traditional databases to provide a complete and organized view of related information.